Behavioral Design Patterns Document

Bank Management System Java application. Behavioral design pattern assignment with class diagrams.

Executive Summary

In this document, I explain the behavioral design pattern problems I found in the Bank Management System and how I solved each one using the correct pattern. Template Method, Mediator, Strategy, Observer, and Iterator are behavioral patterns. Singleton is creational, but I included it with the OverDraft use case because it works together with Observer there.

For each case, I describe the issue, the pattern solution, the class roles, and the UML diagram that represents the implementation.

Design Approach

I focused on the real problem each part of the system needed to solve. Then I matched each problem with the design pattern that best fits the responsibility of the classes involved.

The criteria I used were:

- **Template Method:** a base class must define a stable algorithm skeleton and delegate variable steps to subclasses or hooks.
- **Mediator:** a coordinator must centralize interaction among peer components so peers do not directly orchestrate each other.
- **Strategy:** interchangeable algorithms must share an interface and be replaceable by client code without rewriting the caller.
- **Observer + Singleton:** there must be listener registration/notification, and the event source must enforce a single shared instance.
- **Iterator:** traversal must be exposed through an iterator abstraction that hides the aggregate structure, not just a plain loop.

Each case includes the issue I found, the pattern I used to solve it, the role of each class, and the UML diagram.

Findings

1. Template Method: AbstractTransactionTemplate

Pattern type: behavioral.

Issue I found: deposits, withdrawals, and transfers share a fixed transaction lifecycle: validate input, validate account state, check transaction-specific rules, then apply financial impact. Each transaction type varies only the hook implementations.

Pattern solution: I solved this using Template Method. `AbstractTransactionTemplate` defines the transaction workflow skeleton. In the current source, this implementation appears as `AbstractTransactionProcessor.executeTransaction(...)`, a final template method that fixes the workflow order and delegates variable steps to abstract methods.

Pattern roles

Class	Pattern role	Responsibility in this case

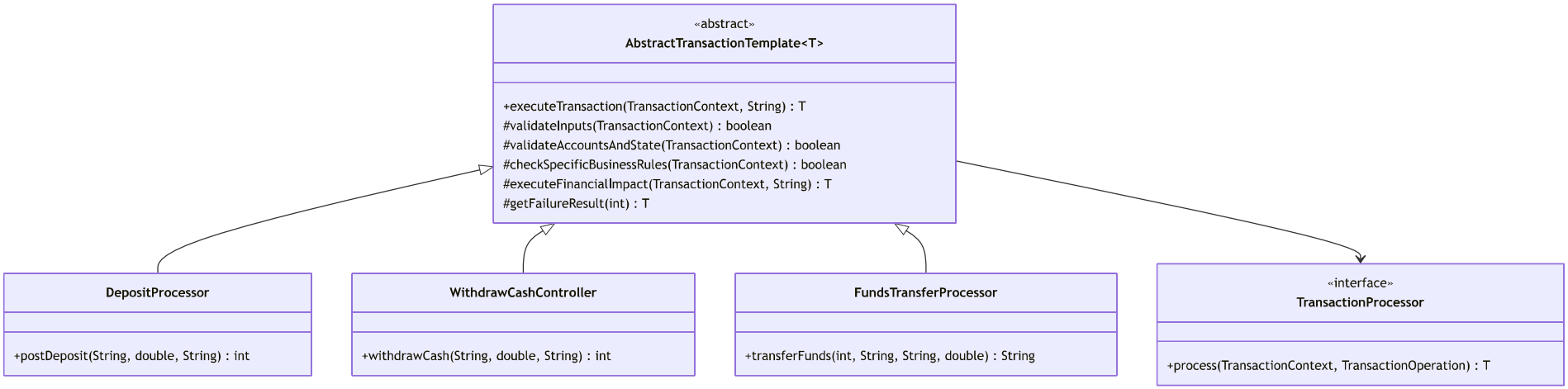
`AbstractTransactionTemplate<T>`
Current source:
`AbstractTransactionProcessor<T>`
	Abstract class / template owner	Defines the fixed transaction workflow and hook methods used by concrete transaction classes.

`DepositProcessor`, `WithdrawCashController`, `FundsTransferProcessor`	Concrete classes	Provide transaction-specific validation, financial impact, transaction recording, and result handling.
`TransactionContext`	Template data context	Carries account, amount, timestamp, transaction id, and error state through the template steps.

Code points:

- `src/main/java/com/bms/domain/controller/AbstractTransactionProcessor.java:32-52` defines final `executeTransaction` and the fixed step sequence.
- `AbstractTransactionProcessor.java:58-78` declares abstract hooks: `validateInputs`, `validateAccountsAndState`, `checkSpecificBusinessRules`, `executeFinancialImpact`, and `getFailureResult`.
- `DepositProcessor.java:55-95`, `WithdrawCashController.java:63-126`, and `FundsTransferProcessor.java:71-169` implement those hooks differently.

```
public final T executeTransaction(...) { ... validateInputs ... validateAccountsAndState ... checkSpecificBusinessRules ... executeFinancialImpact ... }
```



UML class diagram: Template Method for AbstractTransactionTemplate

How this solves it: the base processor prevents transaction workflows from drifting apart while still letting each concrete transaction define its own validation, persistence, and failure semantics.

2. Template Method: AbstractLoanInterestCalculatorTemplate

Pattern type: behavioral.

Issue I found: monthly loan-interest calculation and posting for different account types needs the same workflow but different account selection and interest calculator setup.

Pattern solution: I solved this using Template Method. `AbstractLoanInterestCalculatorTemplate` represents the stable loan-interest workflow. In the current source, this implementation appears as `InterestPostingProcessor.postInterest()` and `processInterest(...)`; subclasses supply `getAccountType()` and the calculator through the superclass constructor.

Pattern roles

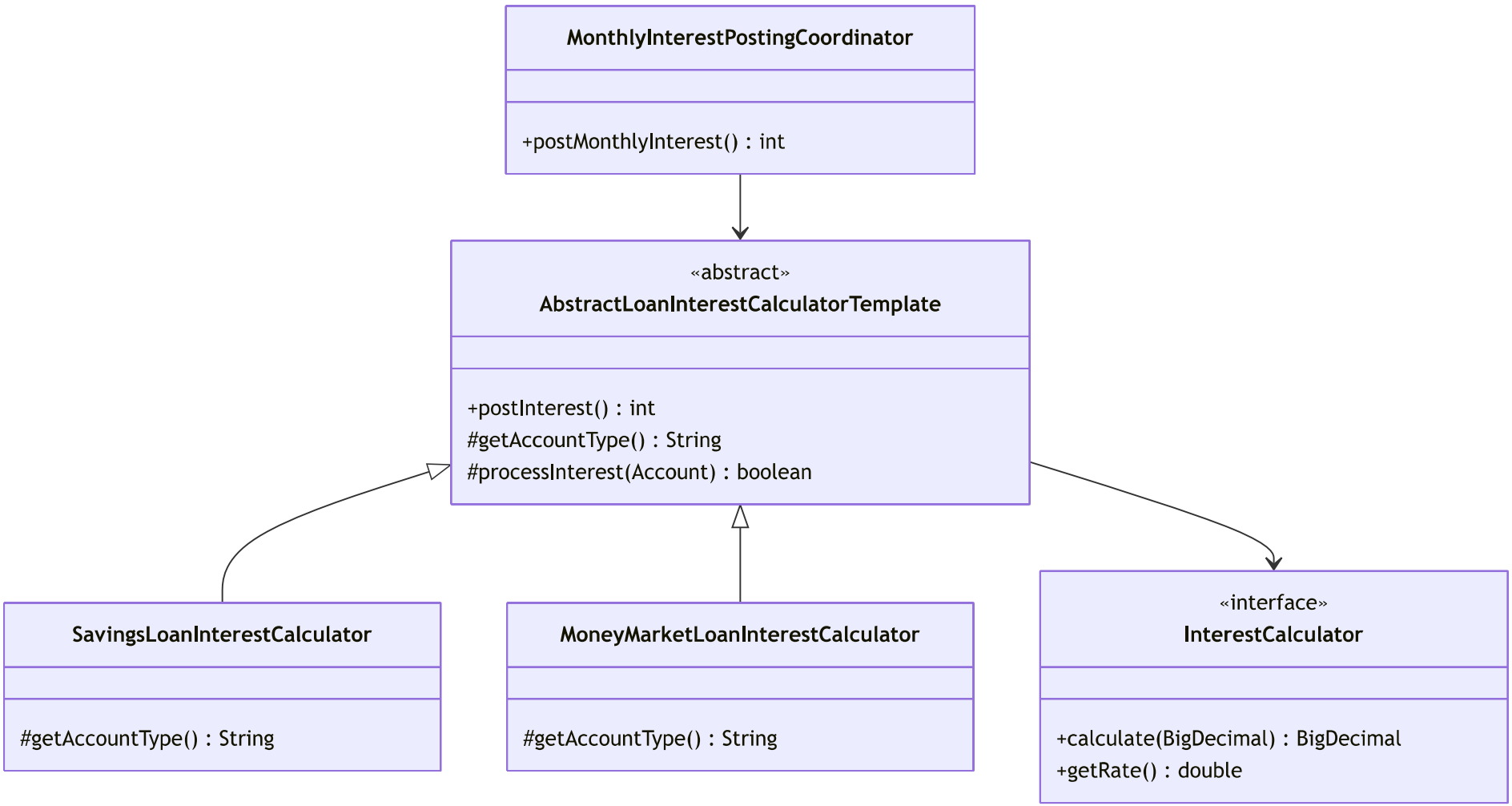
Class	Pattern role	Responsibility in this case

`AbstractLoanInterestCalculatorTemplate`
Current source:
`InterestPostingProcessor`
	Abstract class / template owner	Defines the stable monthly loan-interest calculation/posting algorithm and delegates account-type and calculation variation.

`SavingsLoanInterestCalculator`, `MoneyMarketLoanInterestCalculator`
Current source:
`SavingsInterestPostingProcessor`, `MoneyMarketInterestPostingProcessor`
	Concrete classes	Specialize the template for savings and money-market account interest.

`InterestCalculator`, `SavingsInterest`, `MoneyMarketInterest`	Hook collaborators	Supply the calculation behavior used inside the template workflow.
`MonthlyInterestPostingCoordinator`	Client	Invokes the posting processors without changing the template algorithm.

> - `src/main/java/com/bms/domain/controller/InterestPostingProcessor.java:33-42` loops through accounts for the subclass-defined account type.
> - `InterestPostingProcessor.java:44-70` defines the hook `getAccountType()` and common posting steps: calculate, update account, insert posting, insert transaction.
> - `SavingsInterestPostingProcessor.java:10-17` injects `SavingsInterest` and returns `"Savings"`.
> - `MoneyMarketInterestPostingProcessor.java:10-17` injects `MoneyMarketInterest` and returns `"Money Market"`.



UML class diagram: Template Method for AbstractLoanInterestCalculatorTemplate

How this solves it: new loan-interest variants can reuse the posting algorithm and vary account type and interest calculation without duplicating persistence and transaction-recording code.

3. Mediator: Monthly Interest Rate

Pattern type: behavioral.

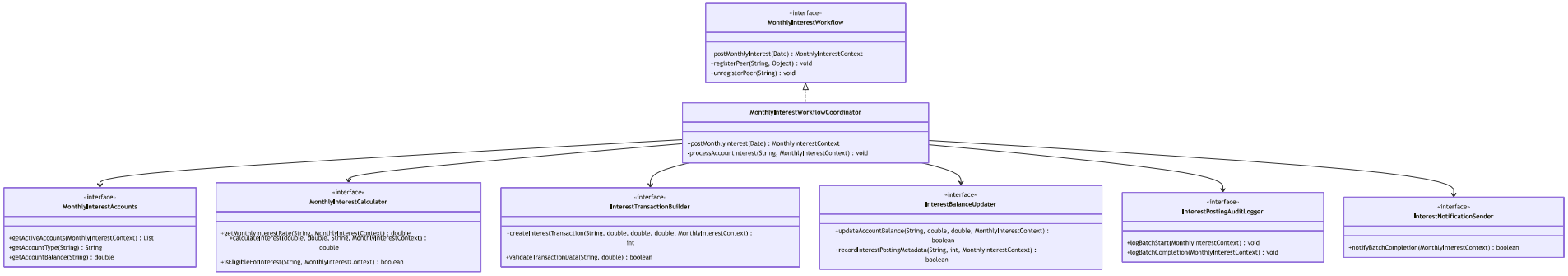
Issue I found: monthly interest processing requires account lookup, eligibility calculation, transaction creation, balance update, audit logging, and notifications. Direct peer-to-peer calls would tightly couple these operations.

Pattern solution: I solved this using Mediator. `MonthlyInterestWorkflowCoordinator` owns references to peer interfaces and centrally controls their interactions in `postMonthlyInterest` and `processAccountInterest`.

Pattern roles

Class	Pattern role	Responsibility in this case
`MonthlyInterestWorkflow`	Mediator interface	Defines the workflow operation exposed to clients.
`MonthlyInterestWorkflowCoordinator`	Concrete mediator	Centralizes account selection, calculation, transaction creation, balance update, audit, and notification ordering.
`MonthlyInterestAccounts`, `MonthlyInterestCalculator`, `InterestTransactionBuilder`, `InterestBalanceUpdater`, `InterestPostingAuditLogger`, `InterestNotificationSender`	Colleague interfaces	Represent peer responsibilities coordinated by the mediator rather than directly coordinating each other.
`DatabaseMonthlyInterestAccounts`, `AccountTypeMonthlyInterestCalculator`, `InterestPostingTransactionBuilder`, `AccountInterestBalanceUpdater`, `ConsoleInterestPostingAuditLogger`, `ConsoleInterestNotificationSender`	Concrete colleagues	Perform the concrete persistence, calculation, update, logging, and notification work requested by the mediator.
`MonthlyInterestContext`	Workflow state	Shares intermediate workflow data between mediator steps.

> - `MonthlyInterestWorkflowCoordinator.java:21-30` stores peer interface references and a registry.
> - `MonthlyInterestWorkflowCoordinator.java:32-45` receives peers by constructor injection.
> - `MonthlyInterestWorkflowCoordinator.java:62-91` orchestrates batch start, active account retrieval, account processing, audit completion, and notification.
> - `MonthlyInterestWorkflowCoordinator.java:112-172` coordinates account lookup, calculator, transaction builder, updater, audit logger, and notification sender.
> - Peer contracts are separated in `MonthlyInterestAccounts.java:7-10`, `MonthlyInterestCalculator.java:4-8`, `InterestTransactionBuilder.java:4-8`, `InterestBalanceUpdater.java:4-8`, `InterestPostingAuditLogger.java:4-10`, and `InterestNotificationSender.java:4-8`.



UML class diagram: Mediator for monthly interest workflow

How this solves it: workflow sequencing lives in one coordinator. The account source, calculator, transaction builder, updater, logger, and notifier can be substituted independently because they do not own the workflow order.

4. Mediator: Fund Transfer

Pattern type: behavioral.

Issue I found: fund transfer validation spans source account, destination account, account status, balance limits, overdraft policy, and transaction recording. The transfer sequence must be centralized to avoid scattered cross-calls.

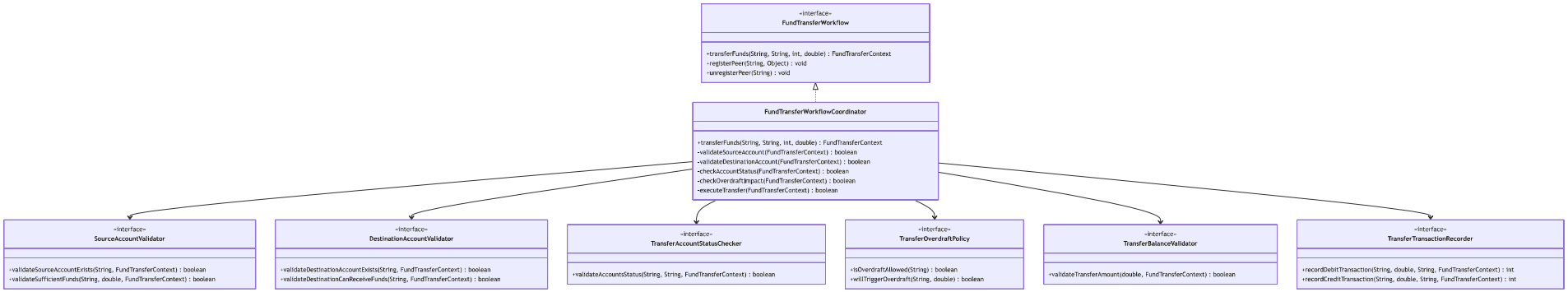
Pattern solution: I solved this using Mediator. `FundTransferWorkflowCoordinator` implements the workflow interface, owns all validation/recording peers, and invokes them in an explicit sequence.

Pattern roles

Class	Pattern role	Responsibility in this case
`FundTransferWorkflow`	Mediator interface	Defines the transfer operation exposed to calling code.
`FundTransferWorkflowCoordinator`	Concrete mediator	Coordinates validators, overdraft policy, balance checks, and transaction recording in one workflow.
`SourceAccountValidator`, `DestinationAccountValidator`, `TransferAccountStatusChecker`, `TransferOverdraftPolicy`, `TransferBalanceValidator`, `TransferTransactionRecorder`	Colleague interfaces	Expose independent transfer responsibilities to the mediator without direct peer-to-peer orchestration.
`SourceAccountAvailabilityValidator`, `DestinationAccountAvailabilityValidator`, `ActiveTransferAccountStatusChecker`, `AccountLimitOverdraftPolicy`, `AvailableFundsTransferValidator`, `DatabaseTransferTransactionRecorder`	Concrete colleagues	Implement the validation, policy, and persistence steps invoked by the mediator.

Class	Pattern role	Responsibility in this case
`FundTransferContext`	Workflow state	Carries account, amount, timestamp, and result data through the transfer workflow.

> - `FundTransferWorkflowCoordinator.java:21-30` holds peer references and a peer registry.
> - `FundTransferWorkflowCoordinator.java:35-48` injects six peer interfaces.
> - `FundTransferWorkflowCoordinator.java:61-125` implements the top-level validation and execution sequence.
> - `FundTransferWorkflowCoordinator.java:130-290` delegates each step to peer interfaces instead of letting peers orchestrate each other.
> - Peer contracts appear in `SourceAccountValidator.java:3-7`, `DestinationAccountValidator.java:5-9`, `TransferAccountStatusChecker.java:5-9`, `TransferOverdraftPolicy.java:4-9`, `TransferBalanceValidator.java:4-9`, and `TransferTransactionRecorder.java:4-8`.



UML class diagram: Mediator for fund transfer workflow

How this solves it: the coordinator makes the transfer validation order explicit and prevents validation components from needing to know each other.

5. Strategy: LoanApprovalStrategy Files

Pattern type: behavioral.

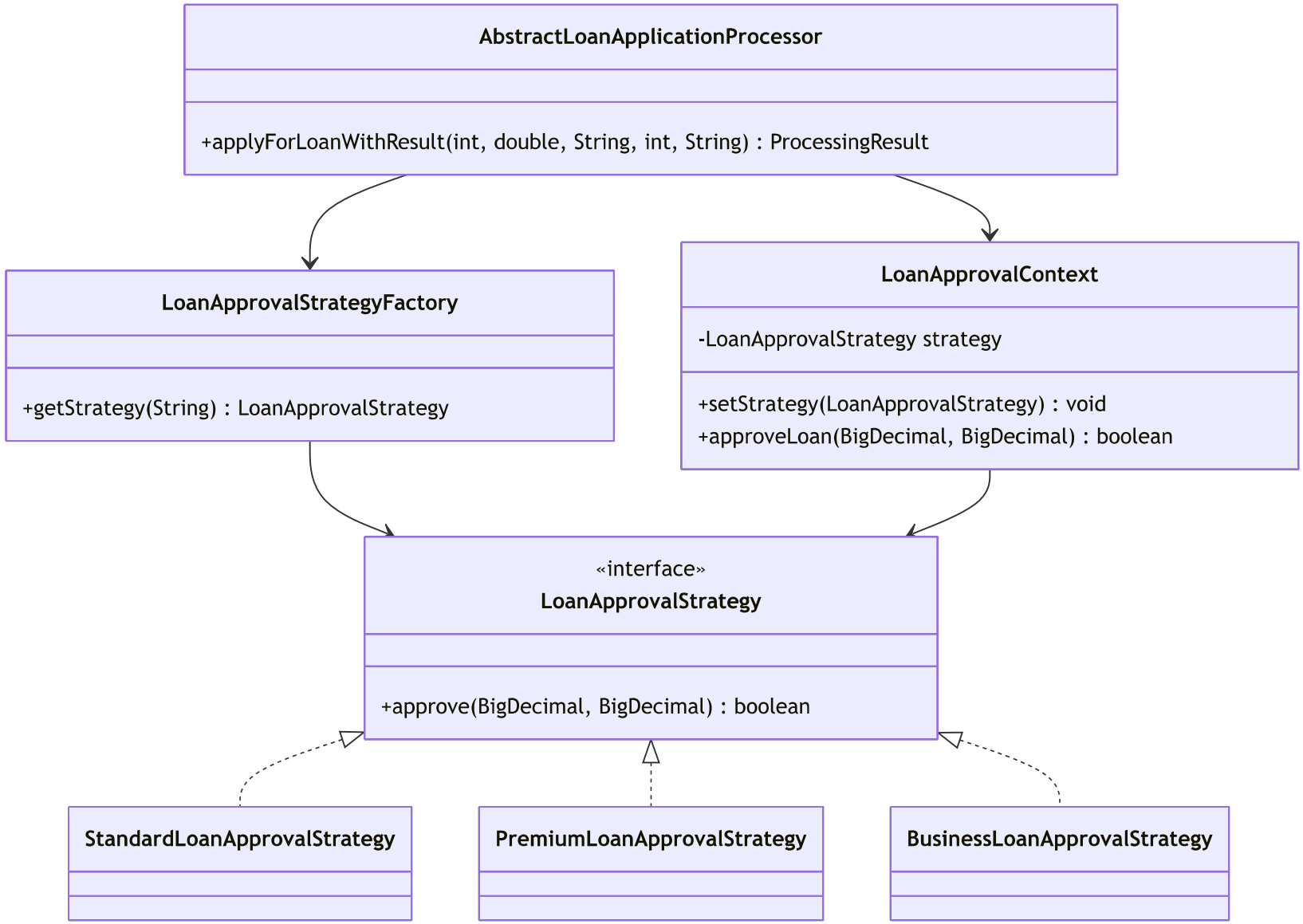
Issue I found: loan approval rules differ by customer tier, but the loan application workflow should not hard-code each approval algorithm.

Pattern solution: I solved this using Strategy. `LoanApprovalStrategy` defines a common approval algorithm interface. In the current source, this implementation appears as the `LoanApprovalPolicy` family. Standard, Premium, and Business strategies implement different approval formulas, while `LoanApprovalContext` delegates to the selected strategy.

Pattern roles

Class	Pattern role	Responsibility in this case
`LoanApprovalStrategy` Current source: `LoanApprovalPolicy`	Strategy interface	Defines the interchangeable loan approval algorithm contract.
`StandardLoanApprovalStrategy`, `PremiumLoanApprovalStrategy`, `BusinessLoanApprovalStrategy` Current source: `StandardLoanApprovalPolicy`, `PremiumLoanApprovalPolicy`, `BusinessLoanApprovalPolicy`	Concrete strategies	Implement distinct approval rules for different customer or loan contexts.
`LoanApprovalContext`	Strategy context	Holds the current policy and delegates approval decisions to it.
`LoanApprovalStrategyFactory` Current source: `LoanApprovalPolicySelector`	Strategy selector	Chooses the appropriate strategy without embedding every approval formula in the caller.
`AbstractLoanApplicationProcessor`	Client	Uses the selected strategy while keeping the application workflow separate from approval formulas.

> - `LoanApprovalPolicy.java:5-7` defines the interchangeable approval operation.
> - `StandardLoanApprovalPolicy.java:5-15`, `PremiumLoanApprovalPolicy.java:5-15`, and `BusinessLoanApprovalPolicy.java:5-17` implement different rules.
> - `LoanApprovalPolicySelector.java:5-20` chooses a strategy by customer type.
> - `LoanApprovalContext.java:7-23` holds and delegates to the selected strategy.
> - `AbstractLoanApplicationProcessor.java:86-91` selects the strategy and asks the context to approve the loan.



UML class diagram: Strategy for LoanApprovalStrategy files

How this solves it: adding a new customer-tier approval rule requires a new policy and selector entry, not rewriting the loan application workflow.

6. Strategy: InterestCalculationStrategy Files

Pattern type: behavioral.

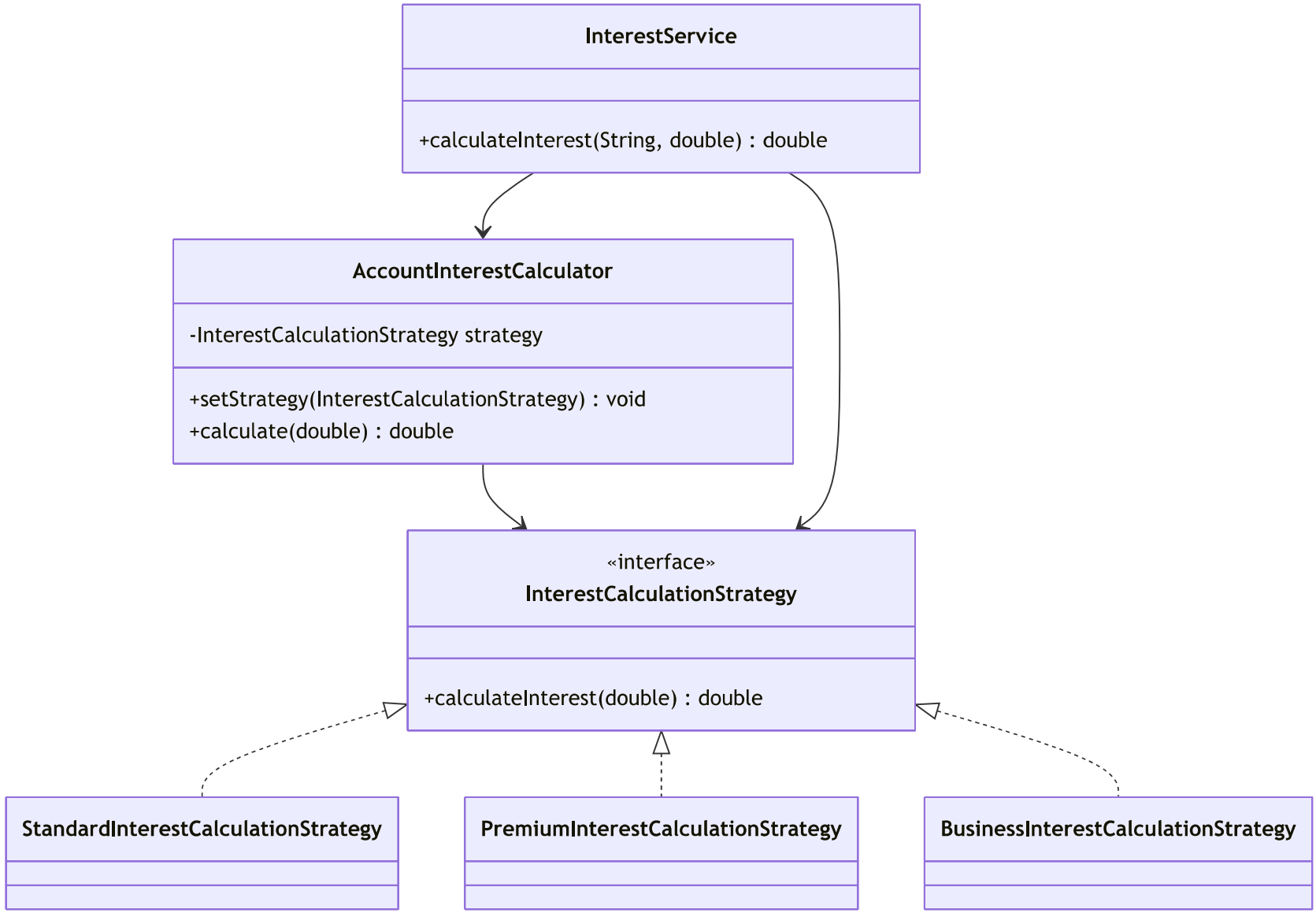
Issue I found: balance interest calculation changes by account/customer type, while caller logic only needs an interest amount.

Pattern solution: I solved this using Strategy. `InterestCalculationStrategy` is a common interest-calculation protocol. In the current source, this implementation appears as the `InterestRatePolicy` family. Concrete strategies implement different calculations and `AccountInterestCalculator` delegates to the active strategy.

Pattern roles

Class	Pattern role	Responsibility in this case
`InterestCalculationStrategy` Current source: `InterestRatePolicy`	Strategy interface	Defines the replaceable interest calculation contract.
`StandardInterestCalculationStrategy`, `PremiumInterestCalculationStrategy`, `BusinessInterestCalculationStrategy` Current source: `StandardInterestRatePolicy`, `PremiumInterestRatePolicy`, `BusinessInterestRatePolicy`	Concrete strategies	Provide alternate interest calculation formulas.
`AccountInterestCalculator`	Strategy context	Delegates interest calculation to the active strategy and allows replacement.
`InterestService`	Client / selector	Selects and applies the strategy appropriate to the account or customer context.

> - `InterestRatePolicy.java:3-5` defines the algorithm interface.
> - `StandardInterestRatePolicy.java:3-8`, `PremiumInterestRatePolicy.java:3-8`, and `BusinessInterestRatePolicy.java:3-8` provide different rates.
> - `AccountInterestCalculator.java:5-17` stores a strategy, allows replacement, and delegates calculation.
> - `InterestService.java:5-20` selects a strategy based on account type and uses the calculator through the common interface.



UML class diagram: Strategy for InterestCalculationStrategy files

How this solves it: the calculation caller is insulated from individual rate formulas and can switch behavior by changing the strategy object.

7. Observer + Singleton: OverDraft Use Case

Pattern type: Observer is behavioral; Singleton is creational.

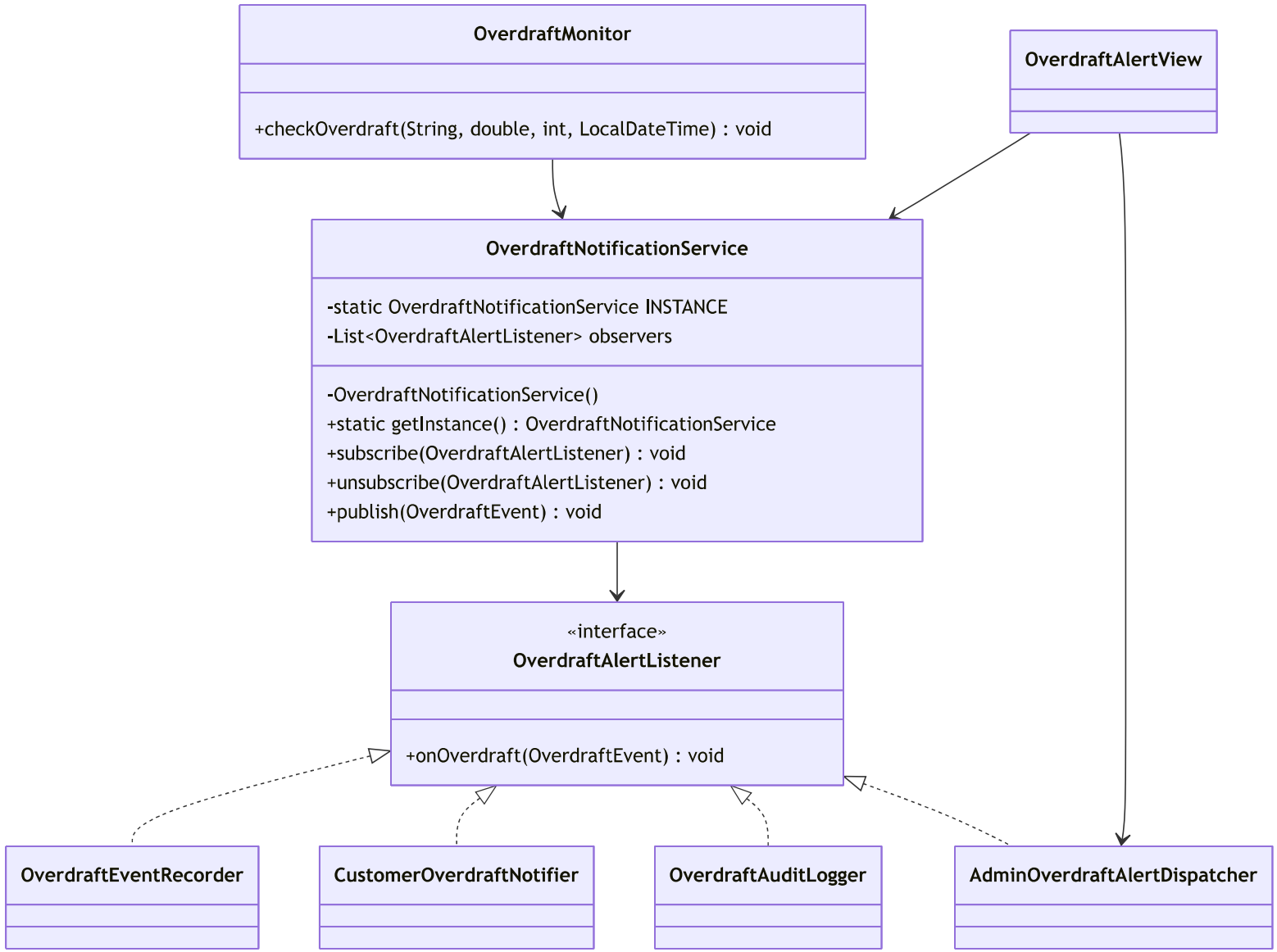
Issue I found: an overdraft event should trigger multiple independent reactions: database recording, customer notification logging, audit logging, and active admin UI refresh. The overdraft detector should not know every reaction.

Pattern solution: I solved this using Observer with Singleton. `OverdraftNotificationService` maintains a listener list, supports subscription and unsubscription, and notifies every listener on publish. It is also an eager Singleton with a private constructor and static instance accessor.

Pattern roles

Class	Pattern role	Responsibility in this case
`OverdraftNotificationService`	Subject / publisher / Singleton	Owns the subscriber list, publishes overdraft events, and exposes one shared instance through `getInstance()`.
`OverdraftAlertListener`	Observer interface	Defines the callback contract for classes interested in overdraft events.
`OverdraftEventRecorder`, `CustomerOverdraftNotifier`, `OverdraftAuditLogger`, `AdminOverdraftAlertDispatcher`	Concrete observers	React to published overdraft events by persisting, notifying, logging, or dispatching admin alerts.
`OverdraftMonitor`	Event source / publisher client	Detects an overdraft condition and publishes the event through the singleton notification service.
`OverdraftAlertView`	Runtime observer client	Subscribes while active so the UI can refresh when a new overdraft event is published.
`OverdraftEvent`	Event payload	Carries account, transaction, amount, timestamp, and alert status data to observers.

> - `OverdraftAlertListener.java:5-7` defines the observer callback.
> - `OverdraftNotificationService.java:11` stores listeners in a list.
> - `OverdraftNotificationService.java:23-31` implements `subscribe` and `unsubscribe`.
> - `OverdraftNotificationService.java:33-41` publishes to each listener through `onOverdraft`.
> - `OverdraftEventRecorder.java:8-25`, `CustomerOverdraftNotifier.java:8-15`, `OverdraftAuditLogger.java:8-18`, and `AdminOverdraftAlertDispatcher.java:7-19` are independent listeners.
> - `OverdraftMonitor.java:35-45` creates an event and publishes it without directly invoking observers.
> - `OverdraftAlertView.java:39-44` subscribes a UI refresh listener, and `OverdraftAlertView.java:84-89` unsubscribes on Back.



UML class diagram: Observer plus Singleton for overdraft notifications

How this solves it: overdraft detection remains focused on detecting and publishing an event. New reactions can be registered without editing the detector.

8. Iterator: Transaction History Filtering

Pattern type: behavioral.

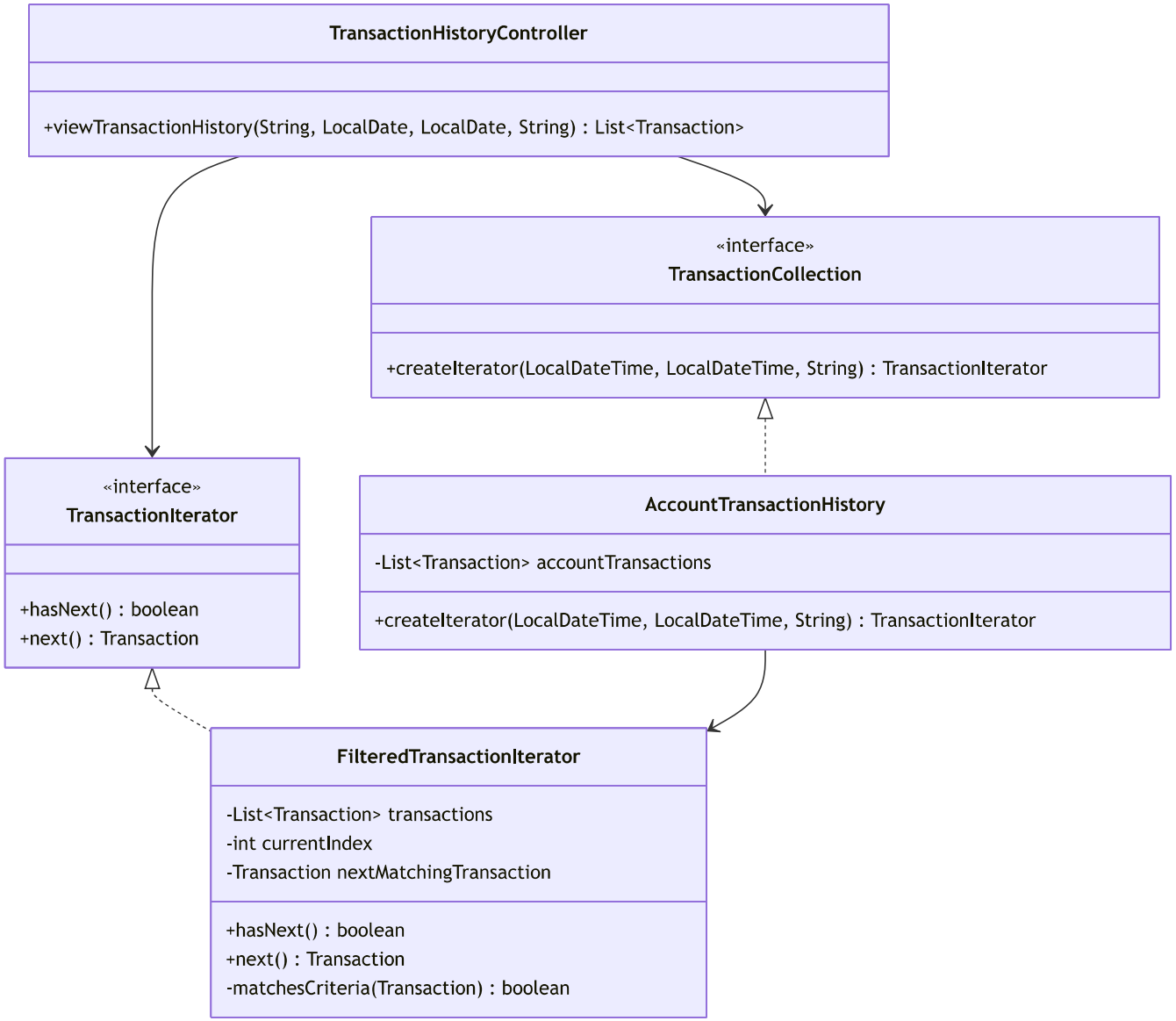
Issue I found: transaction history needs filtered sequential traversal over a list of transactions while hiding traversal and filtering logic from the controller.

Pattern solution: I solved this using Iterator. `TransactionCollection` creates a `TransactionIterator`. `AccountTransactionHistory` hides the internal list. `FilteredTransactionIterator` tracks traversal state and exposes `hasNext` / `next`. The controller consumes the iterator abstraction.

Pattern roles

Class	Pattern role	Responsibility in this case
`TransactionCollection`	Aggregate interface	Exposes iterator creation without exposing the underlying transaction storage.
`AccountTransactionHistory`	Concrete aggregate	Stores transactions and returns a traversal object for account-specific filtering.
`TransactionIterator`	Iterator interface	Defines sequential traversal through `hasNext()` and `next()`.
`FilteredTransactionIterator`	Concrete iterator	Maintains cursor state and applies the account filter while traversing.
`TransactionHistoryController`	Iterator client	Consumes the iterator abstraction rather than the collection internals.
`Transaction`	Element	Represents each item returned during traversal.

> - `TransactionCollection.java:8-17` defines the aggregate method `createIterator`.
> - `TransactionIterator.java:8-19` defines `hasNext` and `next`.
> - `AccountTransactionHistory.java:12-26` wraps the internal list and creates `FilteredTransactionIterator`.
> - `FilteredTransactionIterator.java:14-61` stores traversal state and implements sequential traversal.
> - `FilteredTransactionIterator.java:63-86` hides filter criteria checks inside the iterator.
> - `TransactionHistoryController.java:73-80` uses the aggregate and iterator rather than directly filtering the list.



UML class diagram: Iterator for transaction history filtering

How this solves it: filtering and traversal are separated from transaction-history retrieval, so the controller does not need to know how matching transactions are found.

Observer + Singleton Combination Section

How Observer is used

I used `OverdraftNotificationService` as the Subject/Event Hub. It stores observers in `List<OverdraftAlertListener>`, exposes `subscribe` and `unsubscribe`, and calls `observer.onOverdraft(event)` for each registered listener. The listeners implement a common callback interface and perform separate side effects.

How Singleton is used

The same publisher is an eager Singleton: `OverdraftNotificationService.java:9` declares a private static final instance, `OverdraftNotificationService.java:13-17` has a private constructor, and `OverdraftNotificationService.java:19-21` exposes the shared instance through `getInstance()`. Clients publish and subscribe through `OverdraftNotificationService.getInstance()` in `OverdraftMonitor.java:44` and `OverdraftAlertView.java:43` / `86`.

Conclusion

These are the final design pattern cases I used in the assignment:

- Template Method: 2 cases.
- Mediator: 2 cases.
- Strategy: 2 cases.
- Observer + Singleton: 1 combined case.
- Iterator: 1 case.

Overall, these patterns helped separate responsibilities in the system. Template Method keeps repeated workflows consistent, Mediator centralizes complicated coordination, Strategy separates interchangeable calculations and decisions, Observer with Singleton handles overdraft notifications, and Iterator hides transaction-history traversal details.